

Practica 6

Exercici 1 - Lectura/Escritura de memoria SD

Codi

```
#include <Arduino.h>
#include <SPI.h>
#include <SD.h>
#define PIN_MOSI 11
#define PIN_MISO 13
#define PIN_SCK 12
#define PIN_CS 10

SPIClass spi = SPIClass(FSPI); // o HSPI segons configuració
File myFile;
void setup()
{
  Serial.begin(115200);
  spi.begin(PIN_SCK, PIN_MISO, PIN_MOSI, PIN_CS);
  Serial.print("Iniciando SD ...");
  if (!SD.begin(PIN_CS, spi)) {
    Serial.println("No se pudo inicializar");
    return;
  }
  Serial.println("inicializacion exitosa");
  myFile = SD.open("/archivo.txt");//abrimos el archivo
  if (myFile) {
    Serial.println("archivo.txt:");
    while (myFile.available()) {
      Serial.write(myFile.read());
    }
    myFile.close(); //cerramos el archivo
  } else {
    Serial.println("Error al abrir el archivo");
  }
}
void loop()
{
}
```

Descripció de la sortida del port serie

La sortida que es pot veure al Monitor Sèrie dependrà de si el procés té èxit o falla. En un funcionament correcte, primer es veurà el missatge "Iniciando SD..." seguit immediatament de inicializacion exitosa. A continuació, s'imprimirà "archivo.txt": i, just a sota, començarà a aparèixer tot el contingut de text que hi hagi guardat dins d'aquest fitxer a la targeta SD. Si hi hagués algun problema tècnic, la sortida s'aturaria

amb el missatge "No se pudo inicializar" (si la targeta SD no està ben connectada o formatada) o bé mostraria "Error al abrir el archivo" (si la targeta funciona, però el fitxer "/archivo.txt" no existeix).

Explicació del seu funcionament

Aquest programa té com a objectiu principal connectar-se a un mòdul de targeta SD, llegir un fitxer de text específic i mostrar-ne el contingut a través del port sèrie. Abans de començar la lògica, el codi importa les llibreries necessàries (`SD.h` per a la comunicació maquinari i `SD.h` per gestionar l'arxiu) i defineix quins pins del microcontrolador s'utilitzaran per establir la comunicació SPI (MOSI, MISO, SCK i CS). Com que utilitza el constructor `SPIClass(FSPI)`, és molt probable que aquest codi estigui pensat per a una placa de la família ESP32.

La segona fase té lloc al principi de la funció `setup()`. Primer s'inicia el port sèrie a una velocitat de 115200 bauds perquè puguem veure els missatges a l'ordinador. Seguidament, s'engega el bus SPI amb els pins personalitzats i s'intenta inicialitzar la targeta SD. Aquest pas funciona com un filtre de seguretat: si la targeta no respon (`!SD.begin(...)`), el programa imprimeix un error i utilitza la comanda `retur` per abandonar el procés immediatament, evitant així que el codi intenti llegir un fitxer d'una memòria que no està disponible.

Si la targeta s'ha muntat correctament, entrem en la fase final de lectura. El codi crida la funció `SD.open("/archivo.txt")` per buscar el fitxer. Si l'arxiu s'obre amb èxit, s'inicia un bucle `while` que va llegint el document caràcter per caràcter (`myFile.read()`) i l'envia directament a la pantalla del teu ordinador (`Serial.write()`) fins que s'arriba al final del text. Un cop acabat, és crucial que es tanqui el fitxer amb `myFile.close()` per alliberar memòria. Com que tota aquesta seqüència s'ha programat dins del `setup()`, s'executarà una única vegada quan s'engegui la placa, i per això la funció `loop()` es manté buida.

Exercici 2 - Lectura de etiqueta RFID

Descripció de la sortida del port serie

```
#include <Arduino.h>
#include <SPI.h>
#include <MFRC522.h>

#define SS_PIN 10
#define RST_PIN 9

#define SCK_PIN 12
#define MISO_PIN 13
#define MOSI_PIN 11

MFRC522 mfrc522(SS_PIN, RST_PIN);

void setup() {
  Serial.begin(115200);

  SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, SS_PIN);

  mfrc522.PCD_Init();

  Serial.println("Acerca una tarjeta...");
}

void loop() {
  if (!mfrc522.PICC_IsNewCardPresent()) return;
  if (!mfrc522.PICC_ReadCardSerial()) return;

  Serial.print("UID:");
  for (byte i = 0; i < mfrc522.uid.size; i++) {
    Serial.print(" ");
    Serial.print(mfrc522.uid.uidByte[i], HEX);
  }
  Serial.println();

  mfrc522.PICC_HaltA();
}
```

Descripció de la sortida del port serie

Al Monitor Sèrie, tan bon punt s'iniciï la placa, veuràs un únic missatge inicial: "Acerca una tarjeta....". A partir d'aquí, la pantalla es quedarà en espera de forma silenciosa. Cada vegada que passis una targeta o clauer RFID per sobre del lector, apareixerà el text UID: seguit d'una seqüència de nombres i lletres separats per espais (per exemple, UID: 4A B2 E1 8C). Aquest codi és l'identificador únic de la targeta mostrat en format hexadecimal. Si retires la targeta i la tornes a passar (o en passes una de nova), s'imprimirà una nova línia amb el seu corresponent UID.

Explicació del seu funcionament

Aquest programa té com a finalitat connectar-se a un mòdul lector de radiofreqüència (específicament el model MFRC522) per detectar targetes intel·ligents o clauers NFC i llegir-ne el seu identificador únic (UID). Per aconseguir-ho, primer s'importen les llibreries essencials: `SPI.h` per a la comunicació per maquinari i `MFRC522.h` per controlar directament el sensor. Seguidament, es defineixen els pins que formaran el bus SPI (SCK, MISO, MOSI i SS) juntament amb un pin de reinici (RST) que necessita aquest mòdul. Amb tots aquests pins establerts, es crea l'objecte `mfrc522` que farà de "pont" entre el nostre codi i el maquinari físic.

Dins de la funció `setup()`, es duen a terme els preparatius que s'executen un sol cop en engegar-se. S'inicia la comunicació sèrie a 115200 bauds perquè el microcontrolador pugui parlar amb l'ordinador. Tot seguit, s'activa el bus SPI obligant-lo a utilitzar els pins personalitzats que s'han declarat a dalt, i s'engega el mòdul lector mitjançant la instrucció `mfrc522.PCD_Init()`. Abans de passar a la fase de lectura contínua, s'imprimeix un missatge amistós per avisar l'usuari que el sistema està actiu i a l'espera.

A diferència del codi anterior, aquí la funció `loop()` treballa de valent contínuament. Primer hi trobem dues línies de control vitals: `PICC_IsNewCardPresent()` comprova si hi ha una targeta a l'abast del sensor, i `PICC_ReadCardSerial()` intenta llegir-la. Si no hi ha cap targeta o no es pot llegir, la instrucció `return` fa que s'ignori la resta del codi i el cicle torni a començar de zero. Si la lectura té èxit, s'inicia un bucle `for` que va agafant l'identificador byte per byte i l'imprimeix per pantalla en format hexadecimal (HEX). Finalment, el codi executa `mfrc522.PICC_HaltA()`; aquesta comanda és crucial perquè "adorm" la targeta temporalment, evitant que el programa estigui llegint i saturant la pantalla amb el mateix UID milers de vegades per segon mentre la mantinguis a prop del lector.

Practica 6: Pujada de nota

Codi

Parte 1.- Realizar utilizando el sd y el lector rfid escribiendo en un fichero.log la hora y codigo de cada lectura (describir como se resuelve el hardware para utilizar un spi para dos perifericos).

```
#include <SPI.h>
#include <SD.h>
#include <MFRC522.h>
#include <WiFi.h>
#include <time.h>

#define PIN_MOSI 11
#define PIN_MISO 13
#define PIN_SCK 12

#define CS_SD 10
#define CS_RFID 9
#define RST_RFID 8

MFRC522 rfid(CS_RFID, RST_RFID);

void setup() {
  Serial.begin(115200);

  SPI.begin(PIN_SCK, PIN_MISO, PIN_MOSI);

  pinMode(CS_SD, OUTPUT);
  pinMode(CS_RFID, OUTPUT);

  digitalWrite(CS_SD, HIGH);
  digitalWrite(CS_RFID, HIGH);

  // --- SD ---
  digitalWrite(CS_SD, LOW);
  if (!SD.begin(CS_SD)) {
    Serial.println("Error SD");
    return;
  }
  digitalWrite(CS_SD, HIGH);

  // --- RFID ---
  digitalWrite(CS_RFID, LOW);
  rfid.PCD_Init();
  digitalWrite(CS_RFID, HIGH);

  // --- Hora (NTP) ---
  configTime(3600, 0, "pool.ntp.org");
}
```

```
void loop() {
    digitalWrite(CS_RFID, LOW);

    if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) {
        digitalWrite(CS_RFID, HIGH);
        return;
    }

    String uid = "";
    for (byte i = 0; i < rfid.uid.size; i++) {
        uid += String(rfid.uid.uidByte[i], HEX);
    }

    digitalWrite(CS_RFID, HIGH);

    // Obtener hora
    struct tm timeinfo;
    getLocalTime(&timeinfo);

    char timeStr[30];
    strftime(timeStr, sizeof(timeStr), "%Y-%m-%d %H:%M:%S", &timeinfo);

    // Guardar en SD
    digitalWrite(CS_SD, LOW);

    File file = SD.open("/log.txt", FILE_APPEND);
    if (file) {
        file.print(timeStr);
        file.print(" - UID: ");
        file.println(uid);
        file.close();
        Serial.println("Guardado");
    } else {
        Serial.println("Error archivo");
    }

    digitalWrite(CS_SD, HIGH);

    delay(1000);
}
```

Explicació del codi

Aquest programa evoluciona els codis anteriors integrant un lector RFID, un mòdul de targeta SD i sincronització horària d'Internet (NTP) mitjançant les capacitats Wi-Fi d'una placa com l'ESP32. La gran diferència i complexitat d'aquest disseny és que el lector RFID i la targeta SD comparteixen els mateixos pins del bus SPI (MOSI 11, MISO 13 i SCK 12). Per evitar que els dos dispositius parlin a la vegada i es col·lapsin, el codi utilitza una tècnica de gestió de maquinari manual controlant els pins de selecció de xip (**CS_SD** i **CS_RFID**), activant un dispositiu posant el seu pin en **LOW** i apagant-lo posant-lo en **HIGH**.

Dins del **setup()**, es configuren aquests dos pins de control com a sortides i es posen immediatament en **HIGH** per "enviar a callar" ambdós mòduls abans de començar. A continuació, el programa fa una

inicialització seqüencial: primer activa exclusivament la SD (**CS_SD** en **LOW**), comprova si funciona i la torna a desactivar. Després fa exactament el mateix amb el lector RFID mitjançant **rfid.PCD_Init()**. Per acabar els preparatius, la funció **configTime()** configura el rellotge intern de la placa demanant l'hora real a un servidor d'Internet (**pool.ntp.org**), assumint que la placa ja s'ha connectat prèviament a una xarxa Wi-Fi.

El bloc **loop()** és un cicle continu que es divideix en dues fases gestionades pels pins de selecció. A la primera fase, el microcontrolador activa el lector de targetes (**CS_SD** en **LOW**) i es queda escoltant si hi ha cap targeta a prop. Si no hi ha cap lectura nova, es desconnecta el xip RFID i el codi torna a començar de zero gràcies a la instrucció **return**. Però, si es detecta una targeta amb èxit, el programa extreu el seu identificador únic (UID), el converteix en una cadena de text en format hexadecimal (**String**) i immediatament desactiva el lector RFID per alliberar la línia de comunicació.

A la segona fase del **loop()**, un cop obtingut l'UID, el programa utilitza les estructures de cronometratge internes de C++ (**tm** i **strftime**) per capturar l'hora exacta de la placa i donar-li un format llegible de data i hora (Any-Mes-Dia Hora:Minuts:Segons). Tot seguit, s'apaga el RFID, s'activa la targeta SD (**CS_SD** en **LOW**) i s'obre un fitxer anomenat **/log.txt** en mode d'escriptura addicional (**FILE_APPEND**). Això permet escriure una nova línia amb el registre de l'hora i l'UID detectat sense esborrar el que ja hi havia guardat, creant així un historial complet d'accessos abans de tancar el fitxer i esperar un segon (**delay(1000)**) per a la següent lectura

Parte 2. - Generar una pagina web donde se pueda ver la lectura del lector rfid

```
1970-01-01 01:00:08 - UID: b153f54
1970-01-01 01:00:54 - UID: b153f54
1970-01-01 01:01:03 - UID: b153f54
```

Descripció de la sortida del port serie

La sortida del Monitor Sèrie en aquest programa és molt més minimalista i està pensada exclusivament com un sistema de confirmació en segon pla. Durant l'arrencada, si la targeta SD no està a punt, veuràs el missatge **"Error SD"** i el programa s'aturarà. Si tot s'inicia correctament, la pantalla es quedarà

completament en blanc. No serà fins que apropis una targeta RFID al lector que començaran a passar coses: si el sistema llegeix la targeta i aconsegueix escriure correctament les dades a la memòria, es mostrarà la paraula "Guardado". En cas que la targeta SD falli en aquell moment precís, veuràs el missatge "Error archivo".